



TRAINING PROGRAMME



Lesson 3

Transformation Priority Premise

"As tests grow more specific, the code becomes more generic."— Robert Martin

Transformation Priority Premise

How to evolve the code in TDD

1) Fake implementation

When you hard code exactly the value you need to pass the test

1) Obvious (simple) implementation

When you are sure of the code you need to write. This is what you will be using more often to move forward quickly.

1) Triangulation with the next test

When you want to generalize a behaviour but are not sure how to do it. Starting with fake implementation and then adding more tests will force the code to be more and more generic. Complete one dimension first and then move on the next one with another test case.



Transformation Priority Premise

What is “Obvious implementation” ?

- The table on the next slide shows a list of code evolutions **ordered by complexity**.
- Transformations on the **top of the list** should be preferred to those that are lower.
- When making a test pass, you should try to do so with **simpler transformations** (higher on the list) than with those that are more complex.

<https://blog.cleancoder.com/uncle-bob/2013/05/27/TheTransformationPriorityPremise.html>

* Please do not take this table literally. This is a starting point. Adapt this table to your language and environment.



Transformation Priority Premise - What is “Obvious implementation” ?

#	TRANSFORMATION	STARTING CODE	FINAL CODE
1	<code>{}</code> => <code>nil</code>		<code>return nil</code>
2	<code>nil</code> => <code>constant</code>	<code>return nil</code>	<code>return "1"</code>
3	<code>constant</code> => <code>constant+</code>	<code>return "1"</code>	<code>return "1" + "2"</code>
4	<code>constant</code> => <code>scalar</code>	<code>return "1" + "2"</code>	<code>return argument</code>
5	<code>statement</code> => <code>statements</code>	<code>return argument</code>	<code>return arguments</code>
6	<code>unconditional</code> => <code>conditional</code>	<code>return arguments</code>	<code>if(condition) return arguments</code>
7	<code>scalar</code> => <code>array</code>	<code>dog</code>	<code>[dog, cat]</code>
8	<code>array</code> => <code>container</code>	<code>[dog, cat]</code>	<code>{dog = "DOG", cat = "CAT"}</code>
9	<code>statement</code> => <code>recursion</code>	<code>a + b</code>	<code>a + recursion</code>
10	<code>conditional</code> => <code>loop</code>	<code>if(condition)</code>	<code>while(condition)</code>
11	<code>recursion</code> => <code>tail recursion</code>	<code>a + recursion</code>	<code>recursion</code>
12	<code>expression</code> => <code>function</code>	<code>today - birthday</code>	<code>CalculateAge()</code>
13	<code>variable</code> => <code>mutation</code>	<code>day</code>	<code>var day = 10; day = 11;</code>
14	<code>switch case</code>		



Transformation Priority Premise



Roman numbers kata

Given a positive integer number write a function returning its Roman numeral representation as a String.



Transformation Priority Premise



Arabic	Roman	Arabic	Roman	Arabic	Roman	Arabic	Roman
1	I	8	VIII	60	LX	400	CD
2	II	9	IX	70	LXX	500	D
3	III	10	X	80	LXXX	600	DC
4	IV	20	XX	90	XC	700	DCC
5	V	30	XXX	100	C	800	DCCC
6	VI	40	XL	200	CC	900	CM
7	VII	50	L	300	CCC	1000	M

Arabic	Roman	Thousands	Cents	Tenths	Units
846	DCCCXLVI	-	DCC	XL	VI
1999	MCMXCIX	M	CM	XC	IX
2008	MMVIII	MM	-	-	VIII

